

Project Specification: Automated LeetCode Striver A2Z Tracker

1. Executive Summary & Goal

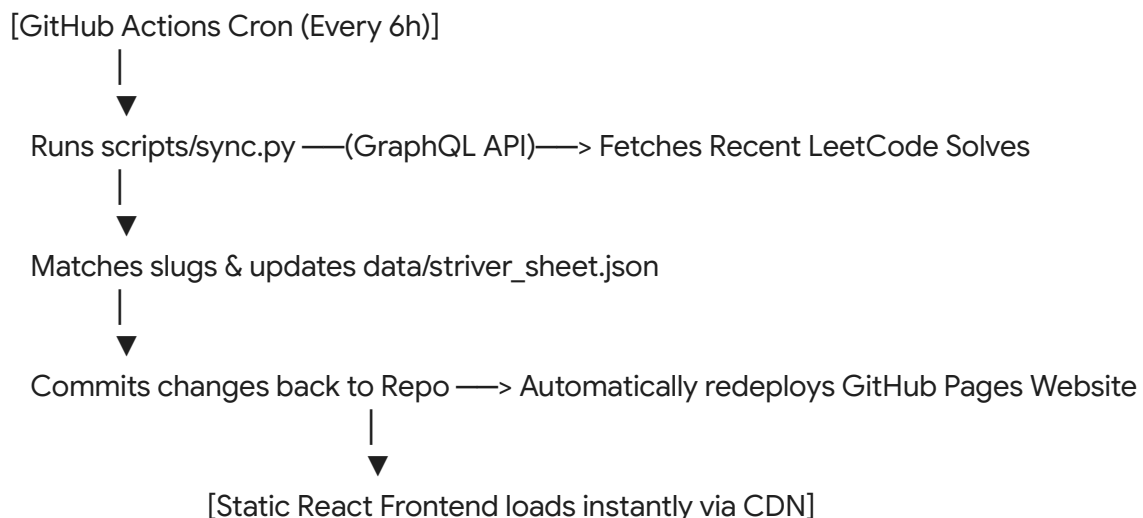
The objective is to build a self-updating, high-performance web dashboard hosted on **GitHub Pages**. The application tracks a user's progress through **Striver's A2Z DSA Sheet** entirely automatically by pulling data directly from LeetCode's internal GraphQL backend.

Key Features:

- **Zero Manual Ticking:** The app automatically identifies solved problems by syncing with the user's LeetCode profile.
- **Historical Tracking:** Displays the exact "last solved" date for every matched problem.
- **Daily Activity Logs:** Displays a daily breakdown of what was solved and includes a GitHub-style contribution heatmap grid.
- **Blazing Fast Performance:** Shakes off any runtime network bottlenecks or API rate limits by utilizing an automated data compilation workflow.

2. Architecture & Data Flow

Because GitHub Pages only hosts static files and blocks direct client-side requests to LeetCode due to **CORS restrictions**, the system uses an **Ahead-of-Time (AOT) Static Data Pipeline**:



1. **The Static Database (data/striver_sheet.json):** A centralized JSON file that acts as the source of truth, storing the structural array of all 470+ problems with their status metadata.
2. **The Automation Engine (scripts/sync.py):** A lightweight Python script executed by **GitHub Actions** on a cron schedule (every 6 hours) or via manual dispatch. It queries LeetCode's public GraphQL endpoint for the user's recent accepted submissions, cross-references them with the JSON file, and mutates matching keys.
3. **The Deployment Layer:** GitHub Actions automatically commits the modified JSON file back into the repository, forcing an instant production rebuild of the static frontend.

3. Core Data Schema (data/striver_sheet.json)

The centralized database must strictly adhere to this uniform array schema:

JSON

```
{
  "step": "Step 1: Learn the basics",
  "section": "Lec 1: Things to know in C++/Java/Python",
  "problemName": "User Input / Output",
  "titleSlug": "user-input-output",
  "leetcodeUrl": "https://leetcode.com/problems/user-input-output/",
  "difficulty": "Easy",
  "completed": true,
  "lastSolved": "2026-06-25"
},
{
  "step": "Step 3: Arrays",
  "section": "Lec 2: Medium Problems",
  "problemName": "Two Sum",
  "titleSlug": "two-sum",
  "leetcodeUrl": "https://leetcode.com/problems/two-sum/",
  "difficulty": "Easy",
  "completed": false,
  "lastSolved": null
}
]
```

4. The Backend Automation Script (scripts/sync.py)

Implement a Python synchronization script using standard libraries (requests, json, datetime, os).

- **Endpoint:** <https://leetcode.com/graphql>
- **Method:** POST
- **Headers:** {"Content-Type": "application/json"}

GraphQL Query Payload:

GraphQL

```
query recentAcSubmissions($username: String!, $limit: Int!) {  
  recentAcSubmissionList(username: $username, limit: $limit) {  
    title  
    titleSlug  
    timestamp  
  }  
}
```

Execution Logic Requirement:

1. Read the target account profile username from an environment variable LEETCODE_USERNAME.
2. Fetch the latest 20 accepted solutions (limit: 20).
3. Parse the local data/striver_sheet.json data array.
4. Convert each submission's Unix timestamp to YYYY-MM-DD local format.
5. If submission.titleSlug == problem.titleSlug, update the matching index item to completed = true and map lastSolved = formatted_date.
6. Save the data back to data/striver_sheet.json with pretty-printed spacing (indent=2).

5. CI/CD Pipeline

(.github/workflows/sync_leetcode.yml)

Create a GitHub Workflow file to execute the cron cycle. Ensure repository settings grant write permissions to workflows.

YAML

name: Sync LeetCode Activity

on:

schedule:

- **cron:** '0 */6 * * *' # Executes every 6 hours

workflow_dispatch: # Allows manual force-sync buttons in the GitHub Actions UI

jobs:

sync:

runs-on: ubuntu-latest

steps:

- **name:** Checkout Repository

uses: actions/checkout@v4

- **name:** Set up Python

uses: actions/setup-python@v5

with:

python-version: '3.10'

- **name:** Install Dependencies

run: pip install requests

- **name:** Run Tracker Sync Script

env:

LEETCODE_USERNAME: "mohak47"

run: python scripts/sync.py

- **name:** Commit and Push Changes

run: |

git config --global user.name "github-actions[bot]"

git config --global user.email "github-actions[bot]@users.noreply.github.com"

git add data/striver_sheet.json

git diff --quiet && git diff --staged --quiet || (git commit -m "chore: automated leetcode sync

[skip ci]" && git push)

6. Frontend UI Requirements (React + TypeScript +

TailwindCSS)

The frontend application must look like a clean modern dashboard, reading data dynamically from the local relative path `./data/striver_sheet.json`.

Component A: Global Progress Analytics

- **Progress Badges:** Shows global completion metrics (e.g., 120 / 475 Solved).
- **Visual Gauges:** Progress bars broken down into Easy (Green), Medium (Yellow), and Hard (Red) sub-components.

Component B: Activity Contribution Heatmap

- A grid component structurally mimicking GitHub's commit contribution matrix.
- Groups completed objects by parsing individual `lastSolved` strings, scaling the block colors darker green depending on the total volume of distinct algorithmic problems solved on that specific date.

Component C: Hierarchical Sheet Accordion Grid

- Group problem structures under clean collapsible folder containers matching Striver's layout (Step 1 $\rightarrow$ Lec 1 $\rightarrow$ rows).
- **Each Row Item Component Must Include:**
 - Problem text name linking to the verified external `leetcodeUrl`.
 - Colored difficulty badge.
 - Status tag (Solved or Unsolved).
 - Visible text string displaying: Last Solved on: YYYY-MM-DD (rendered only if completed).
- **Global Filters:** Add instantaneous global search input bars and toggle switches filtering out already solved questions.

7. Initialization Protocol (Bootstrapping Data)

To ensure the `titleSlug` entries map cleanly without introducing typos, the coding agent must build the foundation file by extracting problems programmatically. The agent should utilize pre-compiled open-source JSON matrices of Striver's sheet, or use a scraper targeting `takeuforward.org/dsa/strivers-a2z-sheet-learn-dsa-a-to-z` to isolate the text names and exact LeetCode routing slugs.